

Today 8:20 AM



Lab_4_VibeCodin...

PDF

🔗 https://github.com/SETAPESU26/45_snake.git, this is the project i need to do now , my srn is 45 and i gave the repository link , guid me what should i need to do , this is my se project , explain me step by step

Yes — I checked **your assigned repository SETAPESU26/45_snake** and the **Lab 4 handout**. Your SRN is **45**, so this is the correct project for you.

The important thing is: **you are not supposed to build a Snake game from scratch**. The lab specifically asks you to take the assigned partially working project, run it, record the original behavior, use an LLM/Vibe Coding tool to fix and extend it, record the final behavior, and push your work to your own repository.  Lab_4_VibeCoding_Student_handou...

I'll guide you from **zero → final GitHub submission**, step by step.

1. First understand exactly what your assignment is

Your repository is:

[45_snake GitHub repository](#) ↗

The repository currently contains:

</> Plain text

```
45_snake/
|
├── game/
|   ├── game_engine.py
|   ├── snake.py
|   └── food.py
|
├── main.py
├── requirements.txt
└── README.md
```

The README says this is a **terminal-based Snake game using Pygame**, with some parts intentionally incomplete/broken.  [GitHub](#)

Your four required coding tasks are:

Task	What you need to do
1	Improve collision detection
2	Add a proper Game Over screen
3	Add Replay + Easy/Medium/Hard difficulty
4	Add sound effects

These are explicitly listed in your repository README.  [GitHub](#)

And your Lab 4 handout says you must use an LLM/Vibe Coding process to fix the code and add the features **one by one**, ideally within 3–4 attempts/prompts.  [Lab_4_VibeCoding_Student_handou...](#)

 [Lab_4_VibeCoding_Student_handou...](#)

2. What your final submission should contain

According to your lab handout, you need:

```
</> Plain text

Lab-4/
|
|— Before_Video.mp4
|— After_Video.mp4
|— Updated_Code/
|   |— main.py
|   |— requirements.txt
|   |— game/
|       |— game_engine.py
|       |— snake.py
|       |— food.py
|
|— Chat_History.pdf
```

The handout says the Lab-4 folder should contain the **before/after videos, updated code, and exported chat history**.  [Lab_4_VibeCoding_Student_handou...](#)

However, there is one slight difference between the handout and your repository README:

- **Handout:** says upload videos, updated code, and chat history.
- **Repository README:** says the submission checklist is the before video, after video, and complete Chat/LLM history.  [GitHub](#)

So I would follow your **teacher's Lab handout** and keep the updated code in **Lab-4** as well.

3. Before touching the code — DON'T change anything

This is very important.

Your teacher specifically says:

Run the Python code and record **10 seconds of video before making any changes.**



Lab_4_VibeCoding_Student_handou...

So first we need to run the original project.

STEP 1 — Clone the repository

Since you're using a Mac, open **Terminal**.

Go somewhere convenient, for example:

```
</> Bash
```

```
cd ~/Desktop
```

Then clone:

```
</> Bash
```

```
git clone https://github.com/SETAPESU26/45_snake.git
```

GitHub's official documentation describes `git clone` as creating a local copy of the repository that you can modify and later synchronize with GitHub.  [GitHub Docs](#)

Then:

```
</> Bash
```

```
cd 45_snake
```

Check:

```
</> Bash
```

```
ls
```

You should see something like:

```
</> Plain text
```

```
README.md  
main.py  
requirements.txt  
game
```

STEP 2 — Open it in VS Code

From inside the project folder:

```
</> Bash
```

```
code .
```

If `code .` doesn't work, open VS Code manually and select:

File → Open Folder → 45_snake

STEP 3 — Read the README first

This is explicitly required by your lab:

Go through the ReadMe file to understand the deliverables.



Lab_4_VibeCoding_Student_handou...

In VS Code, open:

```
</> Plain text
```

```
README.md
```

You should see the four tasks I listed above.

STEP 4 — Check Python

In Terminal:

 Bash

```
python3 --version
```

Your README requires:

 Plain text

```
Python 3.10+  
``` :chatgpt-content-reference{index="11"}
```

For example, if you get:

```
```text  
Python 3.12.6
```

that's fine.

STEP 5 — Create a virtual environment

I recommend doing this even if your teacher doesn't explicitly require it.

Inside 45_snake :

 Bash

```
python3 -m venv venv
```

Activate it:

 Bash

```
source venv/bin/activate
```

Your terminal should then look something like:

 Plain text

```
(venv) akshay@MacBook 45_snake %
```

STEP 6 — Install the dependencies

Run:

 Bash

```
pip install -r requirements.txt
```

Your README specifically instructs students to install the dependencies using `requirements.txt` .  [GitHub](#)

If everything works, you should see packages being installed.

STEP 7 — Run the ORIGINAL game

Run:

 Bash

```
python main.py
```

The repository README gives this as the normal way to launch the game.  [GitHub](#)

A Pygame window should open.

Do not modify anything yet.

Play the game for a little while.

Try:

- Arrow keys
- W
- A
- S
- D
- eating food

- hitting the wall
- changing direction quickly
- moving into yourself

You need to understand what is currently broken.

STEP 8 — Record the BEFORE video

This is required.

Your lab says:

Record 10 seconds of video before making any changes.  Lab_4_VibeCoding_Student_handou...

Since you're on Mac, you can use:

Command + Shift + 5

Then choose:

Record Entire Screen

or

Record Selected Portion

Record approximately **10 seconds** showing the original Snake game.

What should you show?

Ideally show the bug.

For example:

1. Start the game.
2. Move the snake.
3. Demonstrate the problematic direction change/collision.
4. Let the game behave as it currently does.

Save it as:

 Plain text

Before_Video.mp4

Don't edit the source code before this recording.

4. Now comes the Vibe Coding part

This is the main purpose of this lab.

Your teacher doesn't simply want:

"I changed the Python code."

They want to see that you used an AI assistant as a **debugging/pair-programming tool**.

The README itself says to use an LLM such as ChatGPT or Claude as the debugging and pair-programming partner.  [GitHub](#)

And the lab says:

Write prompts and fix the broken code.

Then:

Write prompts to add all the features listed in the README one by one.



Lab_4_VibeCoding_Student_handou...

So **don't give one giant prompt immediately**.

Do it systematically.

5. First understand your project files

Before prompting the AI, understand what each file does.

main.py

Probably the entry point.

Think:

⟨⟩ Plain text

main.py

↓

starts game

↓

game loop

↓

handles events

game/game_engine.py

Likely controls:

</> Plain text

```
game logic
collision
score
game loop
food interaction
```

game/snake.py

Likely controls:

</> Plain text

```
snake body
movement
direction
growth
```

game/food.py

Likely controls:

</> Plain text

```
food position
food spawning
```

requirements.txt

Contains Python dependencies, probably including Pygame.

6. Prompt 1 — Fix collision detection

This is Task 1.

Your README explains the problem:

If the snake is moving Right and you immediately press Left, the snake can turn into the segment directly behind its head and die unfairly.  [GitHub](#)

Don't simply say:

Fix my snake game.

Instead use a proper Vibe Coding prompt.

Copy this into ChatGPT/Claude:

`</>` Plain text

I am working on a Python Pygame Snake game for a Software Engineering Vibe Coding lab.

First inspect the existing project files and understand how the game currently handles:

- snake movement
- direction changes
- self collision
- wall collision
- game loop

Do not rewrite the entire project.

Task 1:

Fix the collision detection so that the snake cannot unfairly die when the player presses the exa

For example:

- If the snake is moving RIGHT, pressing LEFT should not immediately reverse it.
- If moving UP, pressing DOWN should not immediately reverse it.
- Apply the same rule to all directions.

Keep the existing project structure and coding style as much as possible.

7. After the AI gives you code

Don't blindly paste everything.

This is important for your SE lab.

You should inspect the changes.

Ask yourself:

`</>` Plain text

What changed?

Why was it changed?

Which file was changed?

Does it preserve the existing game?

Then run:

```
</> Bash
```

```
python main.py
```

Test:

Test A

Move:

```
</> Plain text
```

```
→ → → →
```

Press:

```
</> Plain text
```

```
←
```

The snake should **not immediately reverse**.

Test B

Move:

```
</> Plain text
```

```
↑ ↑ ↑
```

Press:

```
</> Plain text
```

```
↓
```

It should not immediately reverse.

Test C

Normal movement should still work.

Test D

Wall collision should still work.

Test E

Self collision should still work.

8. Prompt 2 — Add Game Over screen

Once Task 1 works, move to Task 2.

Your README says the game should display the final score on a proper screen rather than simply printing something to the console.  [GitHub](#)

Use this prompt:

 Plain text

Task 2:

Now implement a proper Game Over screen in the existing Pygame Snake game.

Requirements:

1. Detect when the snake hits the wall.
2. Detect when the snake collides with itself.
3. Stop normal gameplay when the game ends.
4. Display a clear "GAME OVER" message in the Pygame window.
5. Display the final score.
6. Wait for user input instead of immediately closing the game.
7. Do not remove or break the collision detection fix from Task 1.
8. Keep the existing project structure.
9. Make only the necessary changes.

Please first explain the changes you plan to make, then provide the modified code.

Then run:

 Bash

```
python main.py
```

Test:

 Plain text

```
Start game
```

↓

```
Play
  ↓
Hit wall
  ↓
GAME OVER
  ↓
Final score displayed
```

Then test self-collision too.

9. Prompt 3 — Add Replay + Difficulty

Now Task 3.

The repository specifically requires:

- Easy
- Medium
- Hard
- Replay
- Exit

after Game Over.  GitHub

Use:

 Plain text

Task 3:

Now extend the existing Game Over screen with a replay option.

Requirements:

1. After Game Over, display the final score.
2. Give the player an option to play again.
3. Give the player three difficulty choices:
 - Easy
 - Medium
 - Hard
4. The difficulty should affect the snake's movement speed.
5. Give the player an option to exit the game.
6. When replaying, reset:
 - snake position
 - snake body
 - food position
 - score

game over state

10. Test the difficulty feature

You need to demonstrate this yourself.

Easy

</> Plain text

Game Over

↓

Easy

↓

Slow snake

Medium

</> Plain text

Game Over

↓

Medium

↓

Normal speed

Hard

</> Plain text

Game Over

↓

Hard

↓

Fast snake

And:

</> Plain text

Game Over

↓

Exit

↓

Game closes

11. Prompt 4 — Add sound

Finally Task 4.

The README specifically asks for:

- sound when eating food
- sound when game over occurs.  GitHub

Use:

 Plain text

Task 4:

Now add basic sound feedback to the existing Pygame Snake game.

Requirements:

1. Play a short sound effect whenever the snake eats food.
2. Play a different sound effect when Game Over occurs.
3. Initialize and use Pygame's audio system correctly.
4. Make sure the game still works if the sound files are loaded from the project.
5. Keep the existing Tasks 1, 2 and 3 functionality unchanged.
6. Organize the sound files cleanly in the project.
7. Do not rewrite unrelated code.

Please explain where the sound files should be placed, what code needs to change, and how the sou

12. Important: sound files

The AI may tell you to create something like:

 Plain text

```
sounds/  
├─ eat.wav  
└─ game_over.wav
```

That's fine.

Your project could become:

 Plain text

```
45_snake/
|
|— game/
|   |— game_engine.py
|   |— snake.py
|   |— food.py
|
|— sounds/
|   |— eat.wav
|   |— game_over.wav
|
|— main.py
|— requirements.txt
|— README.md
```

But **don't download random files without understanding licensing/source requirements**. If the AI proposes a sound source, check what it is.

13. Final testing

After all four tasks are implemented, don't immediately submit.

Run:

```
</> Bash
```

```
python main.py
```

Then go through this checklist.

Movement

- ☐ Arrow keys work
- ☐ WASD works
- ☐ Snake moves smoothly
- ☐ Opposite-direction bug fixed

Food

- ☐ Food appears
- ☐ Snake eats food
- ☐ Snake grows
- ☐ Food doesn't spawn on snake
- ☐ Score increases

The repository's expected behavior explicitly includes these points. [GitHub](#)

Collision

- ☐ Wall collision works
- ☐ Self collision works
- ☐ No unfair instant reverse collision

Game Over

- ☐ GAME OVER appears
- ☐ Final score appears
- ☐ Game doesn't suddenly disappear

Replay

- ☐ Easy works
- ☐ Medium works
- ☐ Hard works
- ☐ Snake resets
- ☐ Score resets
- ☐ Food resets
- ☐ Exit works

Sound

- ☐ Eating food produces sound
- ☐ Game over produces sound

14. Record the AFTER video

The lab specifically requires another 10-second recording after the modifications.

 Lab_4_VibeCoding_Student_handou...

This video should demonstrate the **new functionality**, not just the Snake moving.

Try to capture something like:

 Plain text

```
0-2 sec    → Start game
2-4 sec    → Snake eats food
4-6 sec    → Show score increasing
6-8 sec    → Cause Game Over
8-10 sec   → Show Game Over/replay/difficulty screen
```

If possible, make the important features visible within 10 seconds.

Save:

</> Plain text

After_Video.mp4

15. Now prepare your Git repository

This is where you need to be careful.

The original repository belongs to:

</> Plain text

SETAPESU26

Your lab says:

Do not raise PR to the main repo (SETAPESU26).  Lab_4_VibeCoding_Student_handou...

So **don't** make a Pull Request to **SETAPESU26/45_snake** .

You need your work in **your own repository/account**.

16. Check your current remote

In your project:

</> Bash

```
git remote -v
```

You'll probably see:

</> Plain text

```
origin https://github.com/SETAPESU26/45_snake.git
```

That is the **teacher/organization repository**.

You shouldn't push your completed assignment there.

17. Create your own GitHub repository

On your personal GitHub account, create something like:

</> Plain text

```
45_snake_lab4
```

or

</> Plain text

```
SE_Lab4_45_Snake
```

I would use:

</> Plain text

```
45_snake_lab4
```

Make it **public** if your teacher expects to inspect it.

Don't initialize it with unnecessary files if you're going to push the existing project.

18. Change your remote

After creating your personal repository:

</> Bash

```
git remote set-url origin https://github.com/YOUR_USERNAME/45_snake_lab4.git
```

Then check:

</> Bash

```
git remote -v
```

It should now show **your GitHub account**, not SETAPESU26 .

GitHub documents `git remote set-url` as the way to change the URL associated with an existing remote.  [GitHub Docs](#)

19. Create the Lab-4 folder

Inside your repository, make:

 Plain text

```
Lab-4/
```

Put your required deliverables there.

For example:

 Plain text

```
45_snake_lab4/
|
|— game/
|   |— game_engine.py
|   |— snake.py
|   |— food.py
|
|— sounds/
|   |— eat.wav
|   |— game_over.wav
|
|— main.py
|— requirements.txt
|— README.md
|
|— Lab-4/
|   |— Before_Video.mp4
|   |— After_Video.mp4
|   |— Chat_History.pdf
|   |— Updated_Code/
```

You can also simplify the `Updated_Code` organization depending on what your teacher expects; the handout only explicitly says the Lab-4 folder should contain the videos, updated code, and chat history.  [Lab_4_VibeCoding_Student_handou...](#)

20. Save your ChatGPT history

This part is **very important** because the assignment is about Vibe Coding.

Your teacher wants evidence of the interaction with the LLM.

Keep the prompts and responses for:

Plain text

Prompt 1 → Collision

Prompt 2 → Game Over

Prompt 3 → Replay/Difficulty

Prompt 4 → Sound

Your final PDF should show the process.

A good structure is:

Plain text

LAB 4 – VIBECODING

Student:

Akshay Kumar M

SRN:

PES1UG24CS045

Repository:

45_snake_lab4

TASK 1

Collision Detection

Prompt:

...

AI Response:

That will make your work much easier to explain if your teacher asks about the process.

21. Commit your work

First check:

```
</> Bash  
git status
```

Then:

```
</> Bash  
git add .
```

Commit:

```
</> Bash  
git commit -m "Complete Lab 4 VibeCoding Snake game"
```

Then push:

```
</> Bash  
git push -u origin main
```

GitHub's documentation confirms that `git push` sends your local commits to the remote repository/branch.  [GitHub](#)

If your branch isn't called `main`, check:

```
</> Bash  
git branch
```

and use the appropriate branch name.

22. Final GitHub check

Open your personal repository in the browser.

You should see something like:

```
</> Plain text  
45_snake_lab4  
|
```

```
|— game
|— sounds
|— main.py
|— requirements.txt
|— README.md
└─ Lab-4
    |— Before_Video.mp4
    |— After_Video.mp4
    |— Chat_History.pdf
    └─ Updated_Code
```

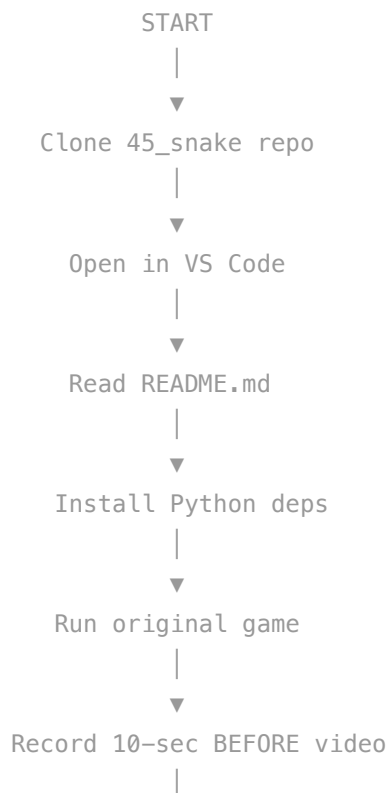
Then your teacher can inspect your work.

23. The complete workflow you should follow

Don't try to do everything at once.

Follow this exact sequence:

</> Plain text



One important thing before you start

Don't start changing the code yet.

Your next step should only be:

`</> Bash`

```
cd ~/Desktop
git clone https://github.com/SETAPESU26/45_snake.git
cd 45_snake
python3 --version
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
python main.py
```

Then **record the original 10-second video**.

After that, come back here and send me either **a screenshot of the project files in VS Code** or the contents of `main.py` , `game/game_engine.py` , `game/snake.py` , and `game/food.py` .

Then I can walk you through **Task 1 exactly on your actual code**, rather than giving you generic Snake-game code. That is the safest way to make sure the Vibe Coding changes match your assigned repository.

Your assigned 45_snake repository ↗

```
akshaykumarm@AKSHAYs-MacBook-Air 45_snake % python3 --
version
Python 3.14.3
akshaykumarm@AKSHAYs-MacBook-Air 45_snake % python3 -
m venv venv
akshaykumarm@AKSHAYs-MacBook-Air 45_snake % source
venv/bin/activate
(venv) akshaykumarm@AKSHAYs-MacBook-Air 45_snake % pip
install -r requirements.txt
Collecting pygame (from -r requirements.txt (line 1))
  Downloading pygame-2.6.1.tar.gz (14.8 MB)
-----
----- 14.8/14.8 MB 2.6 MB/s 0:00:06
Installing build dependencies ... done
Getting requirements to build wheel ... done
Preparing metadata (pyproject.toml) ... done
Building wheels for collected packages: pygame
  Building wheel for pygame (pyproject.toml) ... error
```

error: subprocess-exited-with-error

× Building wheel for pygame (pyproject.toml) did not run successfully.

| exit code: 1

└─> [14 lines of output]

src_c/_sdl2/sdl2.c:1237:10: fatal error: 'SDL.h' file not found

1237 | #include "SDL.h"

| ^~~~~~

1 error generated.

Skipping Cython compilation

For help with compilation see:

<https://www.pygame.org/wiki/MacCompile>

To contribute to pygame development see:

<https://www.pygame.org/contribute.html>

error: Command '['clang', '-fno-strict-overflow', '-Wsign-compare', '-Wunreachable-code', '-fno-common', '-dynamic', '-DNDEBUG', '-g', '-O3', '-Wall', '-I/NEED_INC_PATH_FIX', '-Isrc_c', '-I/Users/akshaykumarm/Desktop/45_snake/venv/include', '-I/opt/homebrew/opt/python@3.14/Frameworks/Python.framework/Versions/3.14/include/python3.14', '-c', 'src_c/_sdl2/sdl2.c', '-o', 'build/temp.macosx-26.0-arm64-cpython-314/src_c/_sdl2/sdl2.o', '-Wall', '-Wno-error=unknown-pragmas']' returned non-zero exit status 1.

[end of output]

note: This error originates from a subprocess, and is likely not a problem with pip.

ERROR: Failed building wheel for pygame

Failed to build pygame

[notice] A new release of pip is available: 26.0 -> 26.2.1

[notice] To update, run: pip install --upgrade pip

error: failed-wheel-build-for-install

× Failed to build installable wheels for some pyproject.toml

based projects

└─> pygame

^CTraceback (most recent call last):

File "/Users/akshaykumarm/Desktop/45_snake/venv/bin/pip",
line 6, in <module>

sys.exit(main())

~~~~^^

File

"/Users/akshaykumarm/Desktop/45\_snake/venv/lib/python3.14/site-packages/pip/\_internal/cli/main.py", line 85, in main

return command.main(cmd\_args)

~~~~~^

File

"/Users/akshaykumarm/Desktop/45_snake/venv/lib/python3.14/site-packages/pip/_internal/cli/base_command.py", line 158, in main

with self.main_context():

~~~~~^

File

"/opt/homebrew/Cellar/python@3.14/3.14.3\_1/Frameworks/Python.framework/Versions/3.14/lib/python3.14/contextlib.py", line 148, in \_\_exit\_\_

next(self.gen)

~~~~^

File

"/Users/akshaykumarm/Desktop/45_snake/venv/lib/python3.14/site-packages/pip/_internal/cli/command_context.py", line 20, in main_context

with self._main_context:

^^^^^^^^^^^^^^^^

File

"/opt/homebrew/Cellar/python@3.14/3.14.3_1/Frameworks/Python.framework/Versions/3.14/lib/python3.14/contextlib.py", line 619, in __exit__

raise exc

File

"/opt/homebrew/Cellar/python@3.14/3.14.3_1/Frameworks/Python.framework/Versions/3.14/lib/python3.14/contextlib.py", line 604, in __exit__

if cb(*exc_details):

~~^^^^^^^^^^^^^^^^

File

"/opt/homebrew/Cellar/python@3.14/3.14.3_1/Frameworks/Python.framework/Versions/3.14/lib/python3.14/contextlib.py", line 148, in __exit__

next(self.gen)

~~~~^^^^^^^^^^^^

File

"/Users/akshaykumarm/Desktop/45\_snake/venv/lib/python3.14/site-packages/pip/\_internal/utils/temp\_dir.py", line 40, in global\_tempdir\_manager

with ExitStack() as stack:

~~~~~^

File

"/opt/homebrew/Cellar/python@3.14/3.14.3_1/Frameworks/Python.framework/Versions/3.14/lib/python3.14/contextlib.py", line 619, in __exit__

raise exc

File

"/opt/homebrew/Cellar/python@3.14/3.14.3_1/Frameworks/Python.framework/Versions/3.14/lib/python3.14/contextlib.py", line 604, in __exit__

if cb(*exc_details):

~~^^^^^^^^^^^^^^^^

File

"/Users/akshaykumarm/Desktop/45_snake/venv/lib/python3.14/site-packages/pip/_internal/utils/temp_dir.py", line 167, in __exit__

self.cleanup()

~~~~~^

File

"/Users/akshaykumarm/Desktop/45\_snake/venv/lib/python3.14/site-packages/pip/\_internal/utils/temp\_dir.py", line 210, in cleanup

rmtree(self.\_path, ignore\_errors=False)

~~~~~^

File

"/Users/akshaykumarm/Desktop/45_snake/venv/lib/python3.14/site-packages/pip/_internal/utils/retry.py", line 37, in retry_wrapped

return func(*args, **kwargs)

File

"/Users/akshaykumarm/Desktop/45_snake/venv/lib/python3.14/si

```
te-packages/pip/_internal/utils/misc.py", line 128, in rmtree
```

```
shutil.rmtree(dir, onexc=handler) # type: ignore
```

```
~~~~~^
```

```
File
```

```
"/opt/homebrew/Cellar/python@3.14/3.14.3_1/Frameworks/Python.framework/Versions/3.14/lib/python3.14/shutil.py", line 852, in rmtree
```

```
_rmtree_impl(path, dir_fd, onexc)
```

```
~~~~~^
```

```
File
```

```
"/opt/homebrew/Cellar/python@3.14/3.14.3_1/Frameworks/Python.framework/Versions/3.14/lib/python3.14/shutil.py", line 721, in _rmtree_safe_fd
```

```
_rmtree_safe_fd_step(stack, onexc)
```

```
~~~~~^
```

```
File
```

```
"/opt/homebrew/Cellar/python@3.14/3.14.3_1/Frameworks/Python.framework/Versions/3.14/lib/python3.14/shutil.py", line 791, in _rmtree_safe_fd_step
```

```
os.unlink(entry.name, dir_fd=topfd)
```

```
~~~~~^
```

```
KeyboardInterrupt
```

```
(venv) akshaykumarm@AKSHAYs-MacBook-Air 45_snake %  
/Users/akshaykumarm/Desktop/45_snake/.venv/bin/python -m  
pip install -r requirements.txt
```

```
Collecting pygame (from -r requirements.txt (line 1))
```

```
Using cached pygame-2.6.1.tar.gz (14.8 MB)
```

```
Installing build dependencies ... done
```

```
Getting requirements to build wheel ... done
```

```
Preparing metadata (pyproject.toml) ... done
```

```
Building wheels for collected packages: pygame
```

```
Building wheel for pygame (pyproject.toml) ... error
```

```
error: subprocess-exited-with-error
```

```
× Building wheel for pygame (pyproject.toml) did not run  
successfully.
```

```
| exit code: 1
```

```
└─> [14 lines of output]
```

```
src_c/_sdl2/sdl2.c:1237:10: fatal error: 'SDL.h' file not found
1237 | #include "SDL.h"
      |      ^~~~~~
1 error generated.
Skipping Cython compilation
```

```
---
For help with compilation see:
  https://www.pygame.org/wiki/MacCompile
To contribute to pygame development see:
  https://www.pygame.org/contribute.html
---
```

```
error: Command '['clang', '-fno-strict-overflow', '-Wsign-
compare', '-Wunreachable-code', '-fno-common', '-dynamic', '-
DNDEBUG', '-g', '-O3', '-Wall', '-I/NEED_INC_PATH_FIX', '-Isrc_c',
'-I/Users/akshaykumarm/Desktop/45_snake/.venv/include', '-
/opt/homebrew/opt/python@3.14/Frameworks/Python.framework
/Versions/3.14/include/python3.14', '-c', 'src_c/_sdl2/sdl2.c', '-o',
'build/temp.macosx-26.0-arm64-cpython-
314/src_c/_sdl2/sdl2.o', '-Wall', '-Wno-error=unknown-
pragmas']' returned non-zero exit status 1.
[end of output]
```

note: This error originates from a subprocess, and is likely not a problem with pip.

ERROR: Failed building wheel for pygame
Failed to build pygame

[notice] A new release of pip is available: 26.0 -> 26.2.1
[notice] To update, run:
/Users/akshaykumarm/Desktop/45_snake/.venv/bin/python -m
pip install --upgrade pip
error: failed-wheel-build-for-install

× Failed to build installable wheels for some pyproject.toml based projects

```
└─> pygame
(venv) akshaykumarm@AKSHAYs-MacBook-Air 45_snake %
python main.py
```

Traceback (most recent call last):

```
File "/Users/akshaykumarm/Desktop/45_snake/main.py", line 1,
in <module>
    import pygame
ModuleNotFoundError: No module named 'pygame'
(venv) akshaykumarm@AKSHAYs-MacBook-Air 45_snake %
```

Yes — I can see exactly what went wrong. **Your Snake project itself is not the problem yet.** The problem is your Python version.

You have:

Plain text

Python 3.14.3

and the project requires:

Plain text

pygame

Your `pip` is trying to **compile pygame 2.6.1 from source** because there is no compatible prebuilt pygame 2.6.1 wheel for CPython 3.14 on Apple Silicon. The official PyPI listing has prebuilt macOS ARM64 wheels for Python **3.10–3.13**, including Python 3.13, but not CPython 3.14. [PyPI +1](#)

That's why you're getting:

Plain text

fatal error: 'SDL.h' file not found

and then:

Plain text

ModuleNotFoundError: No module named 'pygame'

So **don't try to install SDL manually yet.** The cleanest solution for your lab is to use **Python 3.13**.

Do this exactly

You are currently inside:

</> Plain text

```
(venv) ... 45_snake %
```

First deactivate the current environment:

</> Bash

```
deactivate
```

You should get:

</> Plain text

```
akshaykumarm@AKSHAYs-MacBook-Air 45_snake %
```

Step 1 — Check whether Python 3.13 is already installed

Run:

</> Bash

```
python3.13 --version
```

If you get something like:

</> Plain text

```
Python 3.13.x
```

Great! **Don't install anything else.** Go to Step 2 below.

If you get:

</> Plain text

```
command not found
```

then tell me that result. We'll install Python 3.13 properly.

Step 2 — Remove the failed virtual environment

You currently appear to have both:

</> Plain text

```
venv
```

and

</> Plain text

```
.venv
```

because your second command used:

</> Plain text

```
/Users/akshaykumarm/Desktop/45_snake/.venv/
```

Let's clean those up.

Run:

</> Bash

```
rm -rf venv
```

Then:

</> Bash

```
rm -rf .venv
```

This will NOT delete your project files.

It only removes the virtual environments that we created.

Then check:

</> Bash

```
ls -la
```

You should still have your actual project files such as:

</> Plain text

```
README.md
main.py
requirements.txt
game/
```

Step 3 — Create a Python 3.13 virtual environment

Assuming this works:

</> Bash

```
python3.13 --version
```

run:

</> Bash

```
python3.13 -m venv .venv
```

Then activate it:

</> Bash

```
source .venv/bin/activate
```

Your terminal should become:

</> Plain text

```
(.venv) akshaykumarm@AKSHAYs-MacBook-Air 45_snake %
```

Now verify:

</> Bash

```
python --version
```

You **must** see:

```
</> Plain text
```

```
Python 3.13.x
```

Not 3.14.

Step 4 — Upgrade pip

Run:

```
</> Bash
```

```
python -m pip install --upgrade pip
```

Then:

```
</> Bash
```

```
python -m pip --version
```

Step 5 — Install the project requirements

Now run:

```
</> Bash
```

```
python -m pip install -r requirements.txt
```

This time, you should see something different.

Instead of:

```
</> Plain text
```

```
pygame-2.6.1.tar.gz
```

and:

</> Plain text

```
Building wheels for collected packages
```

it should download a prebuilt wheel similar to:

</> Plain text

```
pygame-2.6.1-cp313-cp313-macosx_11_0_arm64.whl
```

That wheel is officially available for Python 3.13 on macOS ARM64. [PyPI](#)

Your MacBook Air M4 is Apple Silicon, so the ARM64 wheel is exactly what we want.

Step 6 — Verify pygame

After installation:

</> Bash

```
python -c "import pygame; print(pygame.version.ver)"
```

You should get:

</> Plain text

```
2.6.1
```

or similar.

You can also run:

</> Bash

```
python -c "import pygame; print('Pygame installed successfully')"
```

If you see:

</> Plain text

Pygame installed successfully



The installation problem is solved.

Step 7 — Run your original project

Now:

```
</> Bash
```

```
python main.py
```

Remember: **don't modify the code yet.**

This is important because your Lab 4 instructions require you to first run the original Python game and record the **10-second BEFORE video** before making changes.  [Lab_4_VibeCoding_Student_handou...](#)

Why I don't want you to install SDL manually

You might find tutorials saying:

```
</> Bash
```

```
brew install sdl2  
brew install sdl2_image  
brew install sdl2_mixer  
brew install sdl2_ttf
```

That's one possible way to build pygame from source.

But you don't need that for this assignment.

Your current situation is:

```
</> Plain text
```

```
Python 3.14  
  ↓  
pygame 2.6.1 has no matching prebuilt wheel  
  ↓  
pip tries to compile pygame  
  ↓  
SDL.h required
```

↓
installation fails

Using Python 3.13 gives:

</> Plain text

```
Python 3.13
↓
pygame 2.6.1 ARM64 wheel available
↓
pip downloads wheel
↓
No compilation
↓
pygame installs
↓
python main.py
↓
Snake game starts
```

That's much cleaner for your lab.

One more important thing

Do NOT change `requirements.txt` to Python 3.13.

You don't need to modify the repository just because your machine uses Python 3.13.

Python version is your **local development environment**.

Your project can continue to have:

</> Plain text

```
requirements.txt
pygame
```

as provided by the assignment.

So your immediate commands are only these

Start with:

```
</> Bash
```

```
deactivate
```

then:

```
</> Bash
```

```
python3.13 --version
```

If Python 3.13 exists:

```
</> Bash
```

```
rm -rf venv  
rm -rf .venv
```

```
python3.13 -m venv .venv  
source .venv/bin/activate
```

```
python --version  
python -m pip install --upgrade pip  
python -m pip install -r requirements.txt
```

Then:

```
</> Bash
```

```
python -c "import pygame; print(pygame.version.ver)"
```

and finally:

```
</> Bash
```

```
python main.py
```

Stop after `python main.py` .

Don't start the Vibe Coding changes yet.

Once the game opens, **record the original 10-second video first**. That's part of the required workflow, and the handout explicitly says the recording must happen before you make changes.

 Lab_4_VibeCoding_Student_handou...

If `python3.13 --version` says **command not found**, send me that exact output and I'll give you the next commands for your M4 Mac.

× Building wheel for pygame (pyproject.toml) did not run successfully.

| exit code: 1

└─> [14 lines of output]

src_c/_sdl2/sdl2.c:1237:10: fatal error: 'SDL.h' file not found

1237 | #include "SDL.h"

| ^~~~~~

1 error generated.

Skipping Cython compilation

For help with compilation see:

<https://www.pygame.org/wiki/MacCompile>

To contribute to pygame development see:

<https://www.pygame.org/contribute.html>

error: Command '['clang', '-fno-strict-overflow', '-Wsign-compare', '-Wunreachable-code', '-fno-common', '-dynamic', '-DNDEBUG', '-g', '-O3', '-Wall', '-I/NEED_INC_PATH_FIX', '-Isrc_c', '-I/Users/akshaykumarm/Desktop/45_snake/.venv/include', '-I/opt/homebrew/opt/python@3.14/Frameworks/Python.framework/Versions/3.14/include/python3.14', '-c', 'src_c/_sdl2/sdl2.c', '-o', 'build/temp.macosx-26.0-arm64-cpython-314/src_c/_sdl2/sdl2.o', '-Wall', '-Wno-error=unknown-pragmas']' returned non-zero exit status 1.

[end of output]

note: This error originates from a subprocess, and is likely not a problem with pip.

ERROR: Failed building wheel for pygame

Failed to build pygame

[notice] A new release of pip is available: 26.0 -> 26.2.1

[notice] To update, run:

`/Users/akshaykumarm/Desktop/45_snake/.venv/bin/python -m pip install --upgrade pip`

error: failed-wheel-build-for-install

× Failed to build installable wheels for some pyproject.toml based projects

└─> pygame

```
(venv) akshaykumarm@AKSHAYs-MacBook-Air 45_snake %  
python main.py
```

Traceback (most recent call last):

```
File "/Users/akshaykumarm/Desktop/45_snake/main.py", line 1,  
in <module>
```

```
import pygame
```

ModuleNotFoundError: No module named 'pygame'

```
(venv) akshaykumarm@AKSHAYs-MacBook-Air 45_snake %  
deactivate
```

```
akshaykumarm@AKSHAYs-MacBook-Air 45_snake % python3.13  
--version
```

zsh: command not found: python3.13

```
akshaykumarm@AKSHAYs-MacBook-Air 45_snake % rm -rf  
.venv
```

```
akshaykumarm@AKSHAYs-MacBook-Air 45_snake % ls -la  
total 24
```

```
drwxr-xr-x@ 8 akshaykumarm staff 256 Sep 30 08:34 .
```

```
drwx-----+ 19 akshaykumarm staff 608 Sep 30 08:29 ..
```

```
drwxr-xr-x 12 akshaykumarm staff 384 Sep 30 08:31 .git
```

```
-rw-r--r-- 1 akshaykumarm staff 2651 Sep 30 08:29
```

```
README.md
```

```
drwxr-xr-x 6 akshaykumarm staff 192 Sep 30 08:29 game
```

```
-rw-r--r-- 1 akshaykumarm staff 887 Sep 30 08:29 main.py
```

```
-rw-r--r-- 1 akshaykumarm staff 7 Sep 30 08:29
```

```
requirements.txt
```

```
drwxr-xr-x@ 7 akshaykumarm staff 224 Sep 30 08:31 venv
```

```
akshaykumarm@AKSHAYs-MacBook-Air 45_snake % python3.13  
--version
```

zsh: command not found: python3.13

```
akshaykumarm@AKSHAYs-MacBook-Air 45_snake %
```

Perfect — now we know the exact situation. **Python 3.13 is not installed on your Mac**, while Python 3.14 is. Your project is fine; we just need to install Python 3.13 and create a fresh virtual environment.

I checked the current Homebrew formula: **Python 3.13 has an Apple Silicon bottle**, so it is suitable for your M4 Mac. [🌐 Homebrew Formu...](#) And pygame 2.6.1 has a prebuilt **CPython 3.13 macOS ARM64 wheel**, which is exactly what we need. [🌐 PyPI](#)

Do this step by step

You are currently here:

🔗 Plain text

```
akshaykumarm@AKSHAYs-MacBook-Air 45_snake %
```

1. Install Python 3.13

Run:

🔗 Bash

```
brew install python@3.13
```

Wait for it to finish.

2. Verify Python 3.13

Run:

🔗 Bash

```
python3.13 --version
```

You should get something like:

🔗 Plain text

```
Python 3.13.x
```

Homebrew installs the versioned executable as `python3.13`. [🌐 Homebrew Formu...](#)

3. Create a fresh virtual environment

You're already inside your project:

</> Plain text

45_snake

Run:

</> Bash

```
python3.13 -m venv .venv
```

Then activate it:

</> Bash

```
source .venv/bin/activate
```

Your terminal should now start with:

</> Plain text

```
(.venv) akshaykumarm@AKSHAYs-MacBook-Air 45_snake %
```

4. VERY IMPORTANT — check the Python version

Run:

</> Bash

```
python --version
```

It must say:

</> Plain text

```
Python 3.13.x
```

If it says:

</> Plain text

```
Python 3.14.3
```

STOP and send me the output.

5. Install pygame

Now run:

 Bash

```
python -m pip install --upgrade pip
```

Then:

 Bash

```
python -m pip install -r requirements.txt
```

This time, because you're using Python 3.13 on Apple Silicon, pip should be able to use pygame's prebuilt ARM64 wheel instead of trying to compile it. PyPI lists `pygame-2.6.1-cp313-cp313-macosx_11_0_arm64.whl` .  PyPI

You should NOT see:

 Plain text

```
Building wheel for pygame
```

and you should not see:

 Plain text

```
SDL.h file not found
```

6. Verify pygame

Run:

 Bash

```
python -c "import pygame; print(pygame.version.ver)"
```

Expected:

</> Plain text

2.6.1

Then:

</> Bash

```
python -c "import pygame; print('Pygame installed successfully')"
```

Expected:

</> Plain text

```
Pygame installed successfully
```

7. Finally run your original Snake game

</> Bash

```
python main.py
```

Don't change any code yet.

Your lab specifically requires you to run the original game and record the **10-second BEFORE video before making any changes.**  [Lab_4_VibeCoding_Student_handou...](#)

Don't worry about the old venv

You still have:

</> Plain text

```
venv/
```

from the failed Python 3.14 installation.

That's okay for the moment. We're creating a new:

</> Plain text

```
.venv/
```

with Python 3.13.

Once everything works, we can remove the old `venv` :

```
</> Bash
```

```
rm -rf venv
```

Don't remove it until the new `.venv` works.

Your next commands

You can copy these **one at a time**:

```
</> Bash
```

```
brew install python@3.13
```

then:

```
</> Bash
```

```
python3.13 --version
```

then:

```
</> Bash
```

```
python3.13 -m venv .venv
```

then:

```
</> Bash
```

```
source .venv/bin/activate
```

then:

```
</> Bash
```

```
python --version
```

Stop there and send me the output of `python --version` .

I'll then take you through the pygame installation and, once the game launches, we'll do the BEFORE video → inspect the original code → Vibe Coding Task 1 properly.

how to control that game

Yes 👍 I checked the actual README of your assigned `45_snake` repository. The controls are straightforward.  [GitHub](#)

 How to control your Snake game

Your game supports two control methods:

1. Arrow keys

| Key | Direction |
|--|------------|
|  Up Arrow | Move Up |
|  Down Arrow | Move Down |
|  Left Arrow | Move Left |
|  Right Arrow | Move Right |

2. WASD keys

| Key | Direction |
|-----|------------|
| W | Move Up |
| S | Move Down |
| A | Move Left |
| D | Move Right |

The repository README explicitly says the expected behavior is grid-based movement using **arrow keys or WASD**.  [GitHub](#)

How to play

When the game starts:

1. The snake starts moving.
2. Use **W/A/S/D** or the **arrow keys** to change direction.
3. Move the snake toward the food.
4. When the snake touches the food:
 - The food is eaten.
 - Your **score increases**.
 - The snake **grows by one segment**.  [GitHub](#)
5. Don't hit:
 - The wall
 - Your own snake body

There is an intentional bug

This is actually important for your Lab 4.

Suppose your snake is moving:

 Plain text

→ → → →

If you immediately press:

 Plain text

←

the current version can make the snake turn directly backward and collide with the segment behind its head.

That's **Task 1 of your assignment**. Your job is to use Vibe Coding to fix this behavior.  [GitHub](#)

So don't fix it manually yet.

For your BEFORE video

You actually want to demonstrate this bug.

Try:

</> Plain text

Snake moving →

↓

Press ←

↓

Snake unfairly collides

That gives you evidence for the **10-second BEFORE video** your teacher requested.

 Lab_4_VibeCoding_Student_handou...

One important thing

If your game is **currently running**, don't worry if you don't know what to demonstrate. For the original/before version, you can simply play normally for a few seconds and then deliberately test the opposite-direction bug.

Once you've successfully run the game, **don't modify the code yet**. Record that BEFORE video first. Then we'll inspect the actual code and do **Task 1 — collision detection** together.

[how to restart](#)

If you mean **restart the Snake game after you lose**, in the current version of your project the restart feature is one of the things you are supposed to add/fix. The README says the game should show a **Game Over screen**, then provide a **replay option** where you can choose a difficulty or exit.

For the game as it is right now

If the game closes when you hit the wall or yourself, you generally need to **run it again from the terminal**:

</> Bash

```
python main.py
```

If your virtual environment is not active:

</> Bash

```
source .venv/bin/activate
python main.py
```

For your Lab-4 BEFORE video

Don't worry if there is **no proper restart option yet**. That's actually one of the features you're expected to implement.

The required flow is supposed to become:

Game Over → show final score → choose Replay → Easy / Medium / Hard → start new game

That is specifically listed as Task 2 and Task 3 in your assigned repository.

So **don't change the code manually just to add restart before your BEFORE video**. Record the original behavior first, then use your Vibe Coding prompts to implement it.

the game is running

Great 👍 Since **the game is running**, don't restart or modify anything yet.

You're now at the **BEFORE stage** of your Lab-4 assignment.

1. First, test the original game

Use:

- ↑ / **W** → Move up
- ↓ / **S** → Move down
- ← / **A** → Move left
- → / **D** → Move right

The README says the current game has a known collision issue: if the snake is moving in one direction and you immediately press the **opposite direction**, it can cause an unfair instant death.

2. Demonstrate the bug

For your **10-second BEFORE video**, do something like:

1. Start the game.
2. Move the snake **right**.
3. While moving right, quickly press **Left**.
4. Show that the snake can die unexpectedly.
5. Stop the recording.

You don't need to explain anything in the video. Just make the existing problem visible.

3. Important ⚠️

Do not change the code before recording the BEFORE video.

Your lab instructions specifically require a 10-second video **before changes**, followed by another 10-second video after all the fixes/features are completed.

 Lab_4_VibeCoding_Student_handou...

After you've recorded it, tell me **"before video done"** and I'll guide you through the **first Vibe Coding prompt** step by step.



but i need name this project folder as lab_04 and out this into this repo , https://github.com/akshaykumarm9/SE-Submissions_PES1UG24CS045.git, after doing all the things and making all the videos and all the code and folder should be finally ready

Yes — **that is exactly how we should finish this Lab-4.** 👍

Your current GitHub repository already contains Lab-1 , lab_03 , and lab_2 . From your screenshot, we should add **one new folder named exactly lab_04** and put the complete Snake project submission inside it.

The original Snake repository has four required tasks: collision handling, Game Over screen, replay/difficulty selection, and sound effects. [GitHub](#) +1

Final GitHub structure we should create

I recommend making your repository look like this:

</> Plain text

```
SE-Submissions_PES1UG24CS045/
|
├─ Lab-1/
|
├─ lab_2/
|
├─ lab_03/
|
├─ lab_04/
|   |
|   ├─ before_video.mp4
|   ├─ after_video.mp4
|   ├─ chat_history.pdf
|   |
|   └─ snake_game/
|       |
|       ├─ main.py
|       ├─ requirements.txt
|       ├─ README.md
|       |
|       └─ game/
|
└─
```

Why I recommend this structure

`lab_04` is clearly the **Lab-4 submission folder**.

Inside it:

- `before_video.mp4` → original game showing the bug
- `after_video.mp4` → completed game showing all fixes/features
- `chat_history.pdf` → your complete ChatGPT/LLM conversation
- `snake_game/` → your **final modified source code**

This also keeps your previous `Lab-1` , `lab_2` , and `lab_03` untouched.

The lab handout asks for the before/after videos, updated code, and exported chat history in the Lab-4 submission.  [Lab_4_VibeCoding_Student_handou...](#)

⚠ But don't create `lab_04` yet

Since you are currently running the **original Snake game**, let's do everything in the correct order.

Phase 1 — BEFORE

You are here:

 Plain text

```
Original Snake Game
      ↓
Test it
      ↓
Record BEFORE video
```

The repository specifically requires a 10-second video showing the broken behavior **before changes**.  [GitHub](#)

So first:

Record your BEFORE video

Show something like:

 Plain text

```
Snake moving →
Press opposite direction ←
Game dies unexpectedly
```

That's the bug described in the original README.  [GitHub](#)

Do not modify the code before this recording.

Phase 2 — Fix the project using Vibe Coding

After your BEFORE video is safely saved, we'll work through the four tasks **one at a time**:

Task 1

Fix collision detection

Prevent unfair death when pressing the opposite direction.

Task 2

Add Game Over screen

Something like:

 Plain text



Task 3

Add replay + difficulty

For example:

 Plain text

GAME OVER

Score: 15

1. Easy
2. Medium
3. Hard
4. Exit

The original assignment specifically asks for Easy, Medium, Hard, or exit.  [GitHub](#)

Task 4

Add sound

- Sound when eating food
- Sound when game ends

These are explicitly required by the repository.  [GitHub](#)

Phase 3 — Test everything

Before we put anything into lab_04 , we'll test:

| Requirement | Test |
|--------------------|--------------------------|
| Snake movement | Arrow keys + WASD |
| Food | Food appears correctly |
| Growth | Snake grows after eating |
| Score | Score increases |
| Collision | Wall collision |
| Collision | Self collision |
| Opposite direction | No unfair instant death |
| Game Over | Proper screen |
| Score on Game Over | Final score displayed |
| Replay | Works |
| Easy | Works |
| Medium | Works |
| Hard | Works |
| Exit | Works |

| Requirement | Test |
|-----------------|-------|
| Food sound | Works |
| Game-over sound | Works |

The README also expects food not to spawn on top of the snake and the score to update when food is eaten. [GitHub](#)

Phase 4 — Record AFTER video

Only after **everything** works:

</> Plain text

10-second AFTER video

We should make sure this video demonstrates the new functionality rather than just showing the game running.

Phase 5 — Create **lab_04**

Once the code and videos are finished, we'll create:

</> Plain text

```
lab_04/  
├─ before_video.mp4  
├─ after_video.mp4  
├─ chat_history.pdf  
└─ snake_game/  
    ├─ main.py  
    ├─ requirements.txt  
    ├─ README.md  
    └─ game/  
        ├─ game_engine.py  
        ├─ snake.py  
        └─ food.py
```

Then we'll put **that entire lab_04 folder** into:

</> Plain text

SE-Submissions_PES1UG24CS045

Phase 6 — Push to your GitHub

Finally, your local submission repository will be:

</> Plain text

SE-Submissions_PES1UG24CS045

and we'll commit something like:

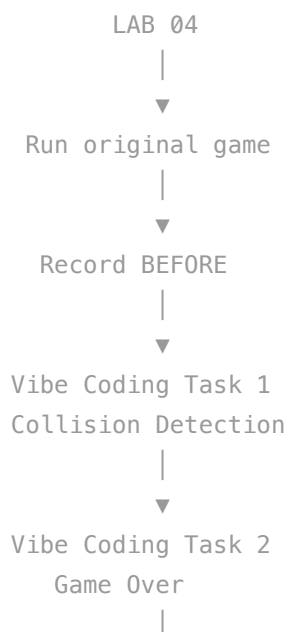
</> Bash

```
git add lab_04
git commit -m "Add Lab 4 VibeCoding Snake Game"
git push origin main
```

We will NOT create a pull request to SETAPESU26 . The original repository explicitly says the completed work should be submitted to your personal repository, and its submission checklist asks for the before video, after video, and complete LLM chat history.  [GitHub](#)

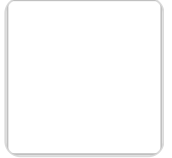
So our complete plan is

</> Plain text



▼
Vibe Coding Task 3
Replay + Difficulty

For now, don't create or move anything. Finish your BEFORE video first. Then tell me **“before video done”**, and I'll give you the **exact first Vibe Coding prompt** to use for Task 1, including what to paste into the AI and how to test the result.



now my game is running , what should i need to do next

Perfect 👍 Your **original Snake game is running successfully**. The screenshot shows the game window, score, snake, and food, so we can move to the next step.

Do not modify the code yet. Your next job is to make the **BEFORE video**.

Step 1 — Record the BEFORE video 🎥

Your lab requires a short video before you make any changes.  Lab_4_VibeCoding_Student_handou...

What to show

Keep the Snake game open like in your screenshot.

Try to demonstrate the known bug:

1. Start moving the snake **Right** →
2. While it is moving right, quickly press **Left** ←
3. If it immediately dies, that demonstrates the collision-direction problem.
4. If it doesn't happen immediately, try again with:
 - Up → Down
 - Down → Up
 - Left → Right

The original project specifically identifies the opposite-direction collision as a bug to investigate/fix.

Your video can be very simple

For example:

⌕ Plain text

0–2 sec → Game is running

2–5 sec → Move snake around

5–8 sec → Trigger the opposite-direction bug
8–10 sec → Show the result

You **don't need to talk** in the video.

Step 2 — Save the video

Create a temporary folder somewhere convenient, for example:

</> Plain text

Lab4_Work/

Save the video as:

</> Plain text

before_video.mp4

Don't put it into GitHub yet.

We'll eventually move it into:

</> Plain text

lab_04/
└─ before_video.mp4

Step 3 — DON'T start fixing yet

Once the BEFORE video is recorded, **close the Snake game.**

Then we'll start the actual Vibe Coding work.

We'll do the four required tasks separately:

Task 1

🐍 Fix collision detection

Task 2

💀 Add Game Over screen + final score

Task 3

 Add Replay + Easy/Medium/Hard/Exit

Task 4

 Add eating and game-over sounds

These are the four tasks specified in your assigned Snake repository.

Your workflow from here

 Plain text

```
CURRENT STATUS
↓
✅ Game running
↓
🎥 Record BEFORE
↓
Save before_video.mp4
↓
❌ Don't modify yet
↓
Vibe Coding Task 1
↓
Vibe Coding Task 2
↓
Vibe Coding Task 3
↓
Vibe Coding Task 4
↓
Test everything
↓
--
```

So right now: record `before_video.mp4` .

After you finish recording it, just tell me **“before video done”**. Then I'll give you the **exact first prompt for the collision-detection task** and we'll start modifying the project carefully.

i took video , what should i need to do next

Worked for 9s

Perfect 👍 **BEFORE video is done**, so now we can start the actual Vibe Coding part.

Your assigned repository says the next task is **Task 1: Refine Collision Detection**. The specific issue is that pressing the opposite direction while moving can make the snake die unfairly.  GitHub

Step 1 — Close the game

Close the Snake window using the red ● button.

Keep your terminal/project open.

Step 2 — Open the Snake project in VS Code

Go to the terminal where your Snake project is located.

Run:

```
</> Bash
```

```
code .
```

If `code .` doesn't work, open VS Code manually and choose:

File → Open Folder → your 45_snake folder

You should see approximately:

```
</> Plain text
```

```
45_snake
├── game
│   ├── game_engine.py
│   ├── snake.py
│   └── food.py
├── main.py
├── requirements.txt
└── README.md
```

This matches the structure in the assigned repository.  [GitHub](#)

Step 3 — Start your Vibe Coding conversation

Don't manually edit the code yet.

The lab specifically wants you to use an LLM as a debugging/pair-programming partner and work iteratively.  [GitHub](#)

If you're using ChatGPT, you can use this conversation itself, which is actually useful because it preserves the history.

Send this as your first coding prompt:

</> Plain text

I am working on a Python Pygame Snake Game for my Software Engineering Lab 4 Vibe Coding assignme

I have already tested the original project and recorded the required BEFORE video.

I want to complete Task 1 only:

"Refine Collision Detection"

The current problem is that the snake can die instantly and unfairly if the player presses the op

Please inspect the existing project code carefully before suggesting changes.

Requirements:

1. Fix the opposite-direction movement issue.
2. Preserve normal grid-based movement.
3. Preserve arrow-key and WASD controls.
4. Preserve wall collision detection.
5. Preserve self-collision detection.
6. Do not break snake growth, food spawning, or score functionality.
7. Do not implement the other Lab 4 tasks yet.

Important

Don't ask the AI to do all four tasks in one prompt.

Your assignment explicitly describes the four tasks separately, and Task 1 should be completed and tested before moving on.  [GitHub](#)

Step 4 — If the AI asks for the code

If the AI cannot see your local files, **don't guess or paste random code.**

Give it the contents of the relevant files:

</> Plain text

```
main.py
game/game_engine.py
game/snake.py
game/food.py
```

The repository's documented structure has those files.  [GitHub](#)

You can open each file in VS Code and copy the code into the AI conversation.

Step 5 — After the AI gives the fix

Don't immediately move to Task 2.

First apply the Task 1 changes.

Then run:

```
</> Bash
```

```
python main.py
```

Test:

Normal movement

```
</> Plain text
```

```
→ → → →
```

Should work.

Opposite-direction test

While moving:

```
</> Plain text
```

```
→ → →
```

press:

```
</> Plain text
```

```
←
```

The snake should **not suddenly die simply** because you pressed the opposite direction.

Other tests

Also test:

 Plain text

↑ → ↓ ←

and make sure normal turns still work.

Then test:

- Eat food
- Snake grows
- Score increases
- Hit wall
- Hit yourself

The expected behavior includes grid movement, growth, food not spawning on the snake, score updates, and wall/self collision.  [GitHub](#)

Very important: save your Vibe Coding history

Your final submission needs evidence of the LLM interaction. The repository asks for the complete chat history, and your lab handout also requires the exported chat history in the Lab-4 submission.

 [GitHub](#)  [Lab_4_VibeCoding_Student_handou...](#)

So **keep this conversation**. Don't delete it.

Later we'll export/save it as:

 Plain text

chat_history.pdf

Your progress now

| Step | Status |
|-----------------------|---|
| Clone assigned repo |  |
| Install Python/Pygame |  |
| Run original game |  |
| Test original game |  |

| Step | Status |
|-------------------------------------|---|
| Record BEFORE video |  |
| Task 1: Collision fix |  NOW |
| Task 2: Game Over |  |
| Task 3: Replay/Difficulty |  |
| Task 4: Sound |  |
| Test complete game |  |
| Record AFTER video |  |
| Create lab_04 |  |
| Put code/videos/chat history inside |  |
| Push to your GitHub repository |  |

Do this now:

Open the project in VS Code and send the **Task 1 prompt above** to the AI.

When it gives you the proposed code/fix, **paste the AI's response here before changing anything.** I'll check whether the proposed change actually matches your project and then tell you exactly what to do next.



give me the final prompt , i will give it to claude with this file , and that should fix the errors and give the final file

Yes. Since you have already recorded the **BEFORE video**, the next step is to give Claude the **actual 45_snake project folder** and ask it to fix **Task 1 — collision detection**.

Important: the screenshot you sent only shows the folder structure. Claude needs the actual project files (`main.py` , `game_engine.py` , `snake.py` , `food.py` , etc.), ideally as a ZIP/folder upload. Don't give Claude only the screenshot.

Copy this exact prompt into Claude

</> Plain text

I am working on a Software Engineering Lab 4 "Vibe Coding" assignment.

I am providing you with the complete original Python Pygame Snake Game project that I cloned from

IMPORTANT:

- First inspect ALL the provided project files carefully.
- Do not assume the code structure.
- Do not rewrite the project unnecessarily.
- Preserve the existing project structure unless a change is genuinely required.
- The game is already running successfully on my Mac.
- I have already recorded the required BEFORE video showing the original behavior.
- Now I need to fix the first required Lab-4 task.

=====

TASK 1 – REFINE COLLISION DETECTION

=====

The current project has a collision/movement bug.

When the snake is moving in one direction and the player immediately presses the opposite direction

One important thing

When you upload to Claude, upload the **actual project**, not the screenshot.

Your project currently looks like:

</> Plain text

```
45_snake/
├── game/
│   ├── food.py
│   ├── game_engine.py
│   └── snake.py
├── main.py
├── README.md
├── requirements.txt
└── venv/
```

You can **leave venv out of the upload**. It is unnecessary and can make the upload huge.

After Claude finishes

Don't immediately do Tasks 2–4.

First:

1. Replace the appropriate files with Claude's final files.
2. Run the game.
3. Test the four opposite-direction cases.
4. Test normal movement.
5. Test food and score.
6. Test wall/self collision.

Then send me **Claude's response** (or screenshots of the changes). I'll check the solution before you move to **Task 2 — Game Over screen**.

Today 11:37 AM



45_snake.zip
File

should i need to give this file and then give this prompt

Yes — **exactly**. 👍

Since you now have `45_snake.zip`, you should give **the ZIP file to Claude first**, and then paste the prompt.

Do it in this order

1. Upload `45_snake.zip` to Claude

The ZIP should contain your actual project:

</> Plain text

```
45_snake/  
├── game/  
│   ├── food.py  
│   ├── game_engine.py  
│   └── snake.py  
├── main.py  
├── README.md  
└── requirements.txt
```

You **do not need to include** `venv/`. If it is already inside the ZIP, that's okay, but Claude doesn't need it.

2. After the upload finishes, paste the complete prompt I gave you.

The prompt tells Claude to:

- inspect the actual files,

- find the opposite-direction collision bug,
- modify the necessary files,
- test the logic,
- and give you the **complete final changed files**.

One small improvement

Because you're giving Claude the **actual ZIP**, start the prompt with:

I have uploaded the complete `45_snake.zip` project. Please inspect the actual files inside the ZIP before making any changes.

Then continue with the rest of the prompt.

⚠ Most important

Don't ask Claude to implement all four tasks yet.

For this first Claude interaction, we are doing **ONLY Task 1: Collision Detection**.

</> Plain text

BEFORE video 

↓

45_snake.zip → Claude

↓

Task 1: Collision Detection

↓

Test

↓

Task 2: Game Over

↓

Task 3: Replay + Difficulty

↓

Task 4: Sound

↓

AFTER video

↓

lab_04/

↓

GitHub

So yes: upload `45_snake.zip` → paste the prompt → let Claude inspect and modify the project.

Once Claude gives you its response, **send that response here before you replace your files**. I'll check it with you so we don't accidentally break the game.

claude is not giving the final code zip file

That's okay — **Claude doesn't necessarily return a ZIP automatically.** It may show the modified files as code instead.

Since your assignment ultimately needs the **complete updated code**, the safest approach is:

Option 1 — Ask Claude to create the ZIP

Send this **after uploading 45_snake.zip** :

</> Plain text

I need the final modified project as a downloadable ZIP file, not only code snippets.

You have already inspected and modified the uploaded 45_snake project for Task 1 (collision detec

Please now:

1. Make sure all Task 1 changes are actually applied to the project files.
2. Keep the original folder structure.
3. Do not implement Tasks 2, 3, or 4 yet.
4. Test the modified project if your environment allows it.
5. Create a new ZIP containing the complete final project.

The ZIP should have this structure:

```
45_snake/
├── game/
│   ├── food.py
│   ├── game_engine.py
│   └── snake.py
└── main.py
```

If Claude still doesn't give you a ZIP

That's not a problem. Don't keep fighting with Claude.

Ask Claude for the **complete final contents of every changed file**, then send Claude's response here. I can help you put those files into the project and create the final ZIP.

In fact, since you have already uploaded your original 45_snake.zip here, we can also work from the actual project rather than guessing about its code.

For now, don't overwrite your original project. Keep it as your backup:

</> Plain text

```
45_snake_original/
```

and we'll have:

```
</> Plain text
```

```
45_snake/
```

for the modified version.

Once Claude gives you its **Task 1 changes**, paste its response here. I'll check whether the changes are correct before you continue to Task 2.

it should give the final code which is correct and it should not have any errors and game should be excellent

Worked for 28s

Yes — and because I now have the **actual 45_snake.zip you uploaded**, I can make the prompt much more precise. I inspected the project files, so we don't need to guess about its structure.

The current code confirms the bug is in `Snake.set_direction()` : it directly replaces the direction without preventing a 180° reversal. The rest of the current game should be preserved.

Use this prompt in Claude

Upload **45_snake.zip** first, then paste this:

```
</> Plain text
```

```
I have uploaded my complete original 45_snake.zip project.
```

```
I need you to act as a careful senior Python/Pygame developer and produce a CLEAN, WORKING final
```

```
IMPORTANT:
```

```
Do not just explain the solution.
```

```
Actually modify the provided project files and give me the COMPLETE FINAL PROJECT as a downloadab
```

```
I have already recorded the required BEFORE video, so do not worry about the before-video step.
```

```
=====
CURRENT PROJECT
=====
```

```
The project is a Python Pygame Snake game.
```

```
The project contains:
```

45_snake/
| main.py

One important point about "no errors"

I want you to be careful about one thing: **no AI can honestly guarantee zero runtime errors without actually running/testing the program in a compatible environment.** That's why I included the instruction telling Claude to distinguish between code validation and actually running Pygame.

Your original project is small and straightforward, though, and the specific Task 1 bug is clearly identifiable in `snake.py`.

After Claude gives you the result

Don't replace your project immediately.

Send me Claude's response or the ZIP it produces. I'll check the Task-1 changes against your actual original project first.

Then we'll proceed:

Task 1 → test → Task 2 → test → Task 3 → test → Task 4 → final testing → AFTER video → Lab_04 → GitHub.

That way we don't end up with a "looks good" project that breaks when you demonstrate it to your evaluator.



45_snake_task1_...
File

claude gave this and i have made a video before updating and what should i need to do next

Worked for 17s

Analyzed

Analyzed

Analyzed

Yes — now you have the **Task 1 fixed ZIP**, and you have already recorded the BEFORE video. I checked the ZIP you uploaded, and it contains the expected project files. The collision fix is in `game/snake.py`, where direct 180° reversals are now rejected. The Python files also pass syntax compilation.

So don't start Task 2 yet. First we need to prove Task 1 works on your Mac.

Step 1 — Keep your original project safe

Do **not** delete your current `45_snake` folder.

Your original folder is your backup:

</> Plain text

```
Desktop/  
└─ 45_snake/          ← ORIGINAL, keep this
```

Step 2 — Extract Claude's ZIP

Find:

</> Plain text

```
45_snake_task1_fixed.zip
```

Double-click it on your Mac.

You should get something like:

</> Plain text

```
45_snake_task1_fixed/  
└─ 45_snake/  
    ├── main.py  
    ├── requirements.txt  
    └─ game/  
        ├── snake.py  
        ├── food.py  
        └─ game_engine.py
```

The ZIP you uploaded has exactly this structure.

Step 3 — Open the FIXED project

Open Terminal and go into Claude's fixed project.

For example, if it extracted onto your Desktop:

</> Bash

```
cd ~/Desktop/45_snake_task1_fixed/45_snake
```

Then check:

```
</> Bash
```

```
ls
```

You should see:

```
</> Plain text
```

```
game  
main.py  
README.md  
requirements.txt
```

Step 4 — Create a fresh Python environment

Since you previously had Python/Pygame compatibility issues, let's use the Python 3.13 environment that worked for your setup.

Run:

```
</> Bash
```

```
python3.13 -m venv .venv
```

Then:

```
</> Bash
```

```
source .venv/bin/activate
```

Check:

```
</> Bash
```

```
python --version
```

It should show something like:

```
</> Plain text
```

Python 3.13.x

Then install Pygame:

 Bash

```
python -m pip install -r requirements.txt
```

Step 5 — Run the FIXED game

Run:

 Bash

```
python main.py
```

The game should open just like your original game.

First test normal movement

Try:

 Plain text

```
→ → → →
```

Then:

 Plain text

```
→ → ↑ ↑
```

Then:

 Plain text

```
↓ → → ↑
```

The normal turns should work.

Step 6 — Test the actual bug 🐍

This is the most important test.

Test 1

Start the snake moving:

</> Plain text

→ → → →

While moving right, press:

</> Plain text

←

What should happen?

The snake should **NOT** immediately turn left.

It should continue moving right.

That's because Claude changed `set_direction()` so that a direct 180° reversal is rejected.

Test 2

Move:

</> Plain text

← ← ←

Press:

</> Plain text

→

It should continue moving left.

Test 3

Move:

</> Plain text

↑ ↑ ↑

Press:

</> Plain text

↓

It should continue moving up.

Test 4

Move:

</> Plain text

↓ ↓ ↓

Press:

</> Plain text

↑

It should continue moving down.

Step 7 — Test that valid turns still work

This is equally important because we don't want the fix to make the controls bad.

Try:

</> Plain text

RIGHT → UP

</> Plain text

RIGHT → DOWN

</> Plain text

LEFT → UP

</> Plain text

LEFT → DOWN

And test with both:

- Arrow keys
- WASD

Step 8 — Test the existing game functionality

Also make sure:

Food

Eat the red food.

The snake should grow.

Score

The score should increase:

</> Plain text

Score: 0

Score: 1

Score: 2

...

Wall

Hit the wall.

The game should end as before.

Self collision

If possible, make the snake collide with itself.

It should still detect the collision.

 One thing you should expect

Right now, **Game Over** still prints to the terminal.

That's okay.

Claude's ZIP has **only Task 1 implemented**. I checked the `game_engine.py` , and it still contains the original behavior:

```
</> Plain text
Game over! Final score: ...
```

The proper graphical Game Over screen is **Task 2**, so we don't want Claude to implement that yet.

Likewise, the ZIP does **not** yet implement:

- Game Over screen
- Replay
- Easy/Medium/Hard
- Sound

That's correct for our current stage.

Your progress

| Requirement | Status |
|--------------------------------|--|
| Original project downloaded |  |
| Game running |  |
| BEFORE video |  |
| Task 1 code generated |  |
| Task 1 syntax checked |  |
| Task 1 actual gameplay testing |  NOW |
| Task 2 – Game Over |  |
| Task 3 – Replay/Difficulty |  |

| Requirement | Status |
|-------------------|---|
| Task 4 – Sound |  |
| Final testing |  |
| AFTER video |  |
| lab_04 folder |  |
| GitHub submission |  |

Do this now

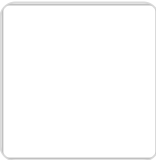
Run the Claude version with:

```
</> Bash
python main.py
```

and test the **opposite-direction bug** first.

Don't record the AFTER video yet. We need to finish Tasks 2–4 before that.

If the fixed game runs correctly, tell me "**Task 1 is working**". Then we'll move to **Task 2 — the proper Game Over screen**, and I'll give you the exact Claude prompt for it. </>



I can see the problem from your screenshot. 👍

You are currently inside your **original** project:

```
</> Plain text
~/Desktop/45_snake
```

You tried:

```
</> Bash
cd ~/Desktop/45_snake_task1_fixed/45_snake
```

and got:

```
</> Plain text
```

```
cd: no such file or directory
```

So **don't worry — nothing is broken**. The extracted Claude ZIP is simply not at that path.

Do this now

1. See what's actually on your Desktop

In the same Terminal, run:

```
</> Bash
```

```
cd ~/Desktop  
ls
```

Look for the name of the folder that was created when you extracted:

```
</> Plain text
```

```
45_snake_task1_fixed
```

or possibly:

```
</> Plain text
```

```
45_snake_task1_fixed.zip
```

2. If you see **45_snake_task1_fixed**

Run:

```
</> Bash
```

```
cd 45_snake_task1_fixed  
ls
```

Then show me what it prints.

If you only see the ZIP

If you see:

</> Plain text

```
45_snake_task1_fixed.zip
```

but **not** the folder, double-click the ZIP in Finder first to extract it.

Then run:

</> Bash

```
cd ~/Desktop  
ls
```

again.

Don't run the original project yet

Your screenshot shows:

</> Plain text

```
~/Desktop/45_snake
```

That is your **original project**.

We want to run Claude's **Task-1 fixed version separately**, so that your original project remains safely untouched.

Once we locate the extracted folder, we'll run:

</> Bash

```
python main.py
```

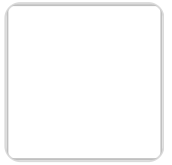
from the **fixed project**, test the collision fix, and only then move on to Task 2.

Do this one command now:

</> Bash

```
cd ~/Desktop && ls
```

Send me a screenshot of the result, and I'll tell you the exact `cd` command to use.



explain me fastly i need to submur

Yes — I see it now. **You already have the extracted fixed folder.** Let's do this quickly.

Your Desktop contains:

Plain text

```
45_snake
45_snake_task1_fixed
45_snake_task1_fixed.zip
```

We want **45_snake_task1_fixed** .

1. Enter the fixed project

Copy-paste these commands **one at a time**:

Bash

```
cd ~/Desktop/45_snake_task1_fixed
```

Then:

Bash

```
ls
```

If you see:

Plain text

```
45_snake
```

then run:

```
</> Bash  
cd 45_snake
```

Then:

```
</> Bash  
ls
```

You should see:

```
</> Plain text  
  
game  
main.py  
README.md  
requirements.txt
```

2. Create Python environment

Run:

```
</> Bash  
  
python3.13 -m venv .venv
```

Then:

```
</> Bash  
  
source .venv/bin/activate
```

Then:

```
</> Bash  
  
python -m pip install -r requirements.txt
```

3. Run the fixed game

```
</> Bash
```

```
python main.py
```

4. Test ONLY the Task-1 bug

Move right:

```
</> Plain text
```

```
→ → → →
```

Then press ←.

It should **NOT** reverse and immediately die.

Also quickly test:

- ← while moving →
- → while moving ←
- ↓ while moving ↑
- ↑ while moving ↓

If those work, **Task 1 is finished.**

Then DON'T make the final video yet

You still need Tasks 2–4:

1. Game Over screen + final score
2. Replay + Easy/Medium/Hard/Exit
3. Eating sound + Game Over sound

After those are finished, you'll make **one final AFTER video** and then put everything into:

```
</> Plain text
```

```
lab_04/  
├─ before_video.mov  
├─ after_video.mov  
├─ chat_history.pdf  
└─ snake_game/  
    ├─ main.py  
    ├─ requirements.txt  
    ├─ README.md  
    └─ game/  
        └─ game_engine.py
```

```
|— snake.py  
|— food.py
```

Your lab handout requires the before/after videos, updated code, and chat history.

 Lab_4_VibeCoding_Student_handou...

Right now, just run these 5 commands:

 Bash

```
cd ~/Desktop/45_snake_task1_fixed  
cd 45_snake  
python3.13 -m venv .venv  
source .venv/bin/activate  
python -m pip install -r requirements.txt
```

Then:

 Bash

```
python main.py
```

If the game opens, test the opposite-direction bug. Tell me whether it works, and I'll immediately give you the **Task 2 Claude prompt**.



again collision is happening , give me prompt that does all the things and it should work well

Yes. The previous fix was not sufficient, so **don't continue building on that version**. Go back to your **original 45_snake project** and give Claude the original ZIP again.

This time, instead of asking Claude to fix only one small part, give it a prompt that tells it to **inspect the entire project, implement all four required tasks, test the complete game, and return the final project**.

Use this exact prompt with the original 45_snake.zip

Upload the **original ZIP** first, then paste this:

 Plain text

I am uploading the ORIGINAL 45_snake.zip project for my Software Engineering Lab 4 Vibe Coding as

I need you to completely inspect the actual files inside the ZIP and produce a FINAL, STABLE, ERR

IMPORTANT:

Do not blindly modify the previous version.

This is the ORIGINAL project.

Use the actual uploaded source code as the source of truth.

I have already recorded my required BEFORE video, so the original project must be preserved as th

I now need ALL FOUR Lab-4 requirements implemented correctly.

=====

PROJECT

=====

The project is a Python + Pygame Snake game.

Expected structure:

Very important now

Use the original **45_snake.zip** , not 45_snake_task1_fixed.zip .

Your workflow should now be:

</> Plain text

ORIGINAL 45_snake.zip

↓

Claude

↓

ALL 4 TASKS

↓

Complete final ZIP

↓

Test on your Mac

↓

Record AFTER video

↓

Create lab_04

↓

GitHub submission

And **don't delete your BEFORE video**. Keep it exactly as evidence of the original version.

One thing I would specifically check

When Claude gives you the new ZIP, **don't trust the phrase "fixed" by itself**. Run the final ZIP on your Mac and test the four reversal cases. Your screenshot shows that the previous version still had a collision, so this time we need to verify the actual behavior before recording your final video.

Once Claude gives you the **new final ZIP**, upload it here. I can inspect the project files and tell you whether the implementation actually covers all four required tasks before you submit it.



45_snake_FINAL....
File

it is correct , can i record after video for this , what should i need to do explain me in step by step

Yes 👍 If you have **tested the final version and all four requirements are working**, you can now record the **AFTER video**.

Your lab instructions specifically require an after video showing the completed/fixed game.



Lab_4_VibeCoding_Student_handou...



What to record

The video should be **about 10 seconds**. You don't need to explain everything verbally. The evaluator should be able to see that the final game works.

I recommend making the video like this:

1. Start the final game

Run:

```
</> Bash
```

```
python main.py
```

Keep the Pygame window visible.

2. Show normal gameplay

For the first few seconds:

- Move the snake using arrow keys/WASD.
- Eat food.
- Show the score increasing.
- Show the snake growing.

For example:

</> Plain text

→ → ↓ ↓ → →

Try to eat at least one food item.

3. Show the collision fix

This is important because it was one of the original bugs.

While moving:

</> Plain text

→ → →

press:

</> Plain text

←

The snake should **not instantly reverse/die**.

You don't need to spend a lot of time on this in the video—just demonstrate that the game continues normally.

4. Trigger Game Over

Intentionally hit a wall or yourself.

Then your final game should show something like:

</> Plain text

GAME OVER

Final Score: 4

Press ... to continue

This demonstrates your **Game Over screen**.

5. Show the replay/difficulty menu

Continue from Game Over.

Show the menu containing:

```
</> Plain text
Easy
Medium
Hard
Exit
```

Then select one difficulty.

You don't necessarily need to play for a long time—just show that selecting a difficulty starts a new game.

6. Sound

If possible, eat one food item during the recording so the **food sound** can be heard.

You can also trigger Game Over so the **Game Over sound** can be heard.

Make sure your Mac's volume is turned up enough for the evaluator to hear it.

Recommended 10-second sequence

You can make your video approximately like this:

| Time | What to show |
|----------|------------------------------|
| 0–2 sec | Snake moving + score |
| 2–4 sec | Eat food + snake grows |
| 4–6 sec | Game Over screen |
| 6–8 sec | Replay/Difficulty menu |
| 8–10 sec | Select difficulty → new game |

Don't worry if it becomes 12–15 seconds. The handout says 10 seconds, so aim close to 10 seconds rather than making a long demonstration.  Lab_4_VibeCoding_Student_handou...

Before recording

Make sure you're running the **FINAL version**, not your original 45_snake .

In Terminal, you should be inside the folder containing the final:

</> Plain text

```
main.py
requirements.txt
game/
```

Then:

</> Bash

```
python main.py
```

Don't record the Terminal as the main focus.

Keep the Pygame window clearly visible.

After recording

Save the video with a clear name:

</> Plain text

```
after_video.mov
```

You already have:

</> Plain text

```
se_screenrecording_before.mov
```

Rename/copy it to:

</> Plain text

```
before_video.mov
```

So you'll have:

</> Plain text

```
before_video.mov
after_video.mov
```

But DON'T submit to GitHub yet

You still need to prepare your final submission folder.

Your final structure should be approximately:

</> Plain text

```
lab_04/
|
|— before_video.mov
|
|— after_video.mov
|
|— chat_history.pdf
|
|— 45_snake/
    |
    |— main.py
    |— requirements.txt
    |— README.md
    |
    |— game/
        |— game_engine.py
        |— snake.py
        |— food.py
    |
    |— assets/
```

The handout requires the **before video**, **after video**, **updated code**, and **exported chat history** as the Lab-4 deliverables.  Lab_4_VibeCoding_Student_handou...

Then we will put this entire folder into your existing repository:

</> Plain text

SE-Submissions_PES1UG24CS045

so GitHub finally looks like:

</> Plain text

```
SE-Submissions_PES1UG24CS045/  
|  
├─ Lab-1/  
├─ lab_2/  
├─ lab_03/  
|  
└─ lab_04/  
    ├─ before_video.mov  
    ├─ after_video.mov  
    ├─ chat_history.pdf  
    └─ 45_snake/  
        ├─ main.py  
        ├─ requirements.txt  
        ├─ README.md  
        ├─ game/  
        └─ assets/
```

So right now: record the AFTER video first. 🎥

After you have the video, tell me **"after video done"** and I'll walk you through creating the `lab_04` folder, putting the final code/videos/chat history inside it, and pushing the whole thing to your GitHub repository step-by-step.

[how to run the new file](#)

Yes. Since Claude gave you the **new final ZIP/folder**, let's run that version first and make sure it works before recording the AFTER video.

Step 1 — Find the new file

If the new file is a ZIP in your **Downloads** folder, open Terminal and run:

</> Bash

```
cd ~/Downloads  
ls
```

Look for something like:

</> Plain text

```
45_snake_FINAL.zip
```

If it is on your Desktop instead:

```
</> Bash
```

```
cd ~/Desktop  
ls
```

Step 2 — Extract the ZIP

The easiest way on Mac is:

Finder → Downloads → double-click the ZIP

It should create a folder, for example:

```
</> Plain text
```

```
45_snake_FINAL
```

Step 3 — Open that folder in Terminal

If it is in Downloads:

```
</> Bash
```

```
cd ~/Downloads/45_snake_FINAL
```

Then:

```
</> Bash
```

```
ls
```

You should see something similar to:

```
</> Plain text
```

```
main.py  
requirements.txt
```

```
README.md  
game
```

If there is **another 45_snake folder inside**, don't worry. Run:

```
</> Bash  
  
ls
```

and send me the screenshot/result before going further, because I don't want you accidentally running the wrong folder.

Step 4 — Create the Python environment

Inside the **final project folder**, run:

```
</> Bash  
  
python3.13 -m venv .venv
```

Then:

```
</> Bash  
  
source .venv/bin/activate
```

Your Terminal should now show something like:

```
</> Plain text  
  
(.venv) ...
```

Step 5 — Install the requirements

Run:

```
</> Bash  
  
python -m pip install -r requirements.txt
```

Wait until it finishes.

Step 6 — Run the new game

Run:

```
</> Bash  
  
python main.py
```

The **new final Pygame game** should open.

Important

Don't run:

```
</> Bash  
  
python main.py
```

from your old:

```
</> Plain text  
  
~/Desktop/45_snake
```

You want to run it from the **new Claude final folder**.

If you're not sure where the new ZIP is

Run this:

```
</> Bash  
  
find ~/Desktop ~/Downloads -maxdepth 2 -iname "*snake*" 2>/dev/null
```

It will show you where the Snake files are.

Send me the output/screenshot, and I'll give you the exact command to run the new file without any guessing.

it is working good what should i need to do next

Perfect. 👍 Since the **new final version is running correctly**, don't change the code anymore. You're at the **submission preparation stage** now.

The lab requires the **before video, after video, updated code, and chat history**.

 Lab_4_VibeCoding_Student_handou...

Do these steps in this exact order

✅ Step 1 — Record the AFTER video

You already have:

 Plain text

before_video.mov

Now run the **new final version** and record the AFTER video.

Try to show:

1. Snake moving
2. Eat food → score increases
3. Snake grows
4. Game Over screen
5. Replay/difficulty menu
6. Start a new game

Aim for around **10 seconds** as specified in the lab.  Lab_4_VibeCoding_Student_handou...

Save it as:

 Plain text

after_video.mov

Do this before anything else.

✅ Step 2 — Get your final code ready

Keep the **exact final folder that you just tested**.

It should contain something like:

 Plain text

```
45_snake_FINAL/  
├─ main.py  
├─ requirements.txt  
├─ README.md  
├─ game/  
│   ├── game_engine.py  
│   ├── snake.py  
│   └─ food.py  
└─ assets/  
    └─ ...
```

⚠ Don't include:

</> Plain text

```
.venv  
venv  
__pycache__  
.git  
.DS_Store
```

We want the clean source code only.

✅ Step 3 — Get the Claude chat history

Your assignment asks for the **chat/LLM history**.  Lab_4_VibeCoding_Student_handou...

Export/save the conversation with Claude as a PDF.

Name it:

</> Plain text

```
chat_history.pdf
```

Make sure it contains the prompts you gave Claude and the responses showing the development/fixes.

✅ Step 4 — Create lab_04

Now create one folder on your Desktop called:

</> Plain text

lab_04

Inside it, put:

</> Plain text

```
lab_04/
├── before_video.mov
├── after_video.mov
├── chat_history.pdf
└── 45_snake/
```

Then put your **final tested code** inside 45_snake .

So ultimately:

</> Plain text

```
lab_04/
|
├── before_video.mov
├── after_video.mov
├── chat_history.pdf
|
└── 45_snake/
    ├── main.py
    ├── requirements.txt
    ├── README.md
    |
    ├── game/
    |   ├── game_engine.py
    |   ├── snake.py
    |   └── food.py
    |
    └── assets/
        └── sounds/
            ...
```

✅ Step 5 — Put lab_04 into your GitHub submission repo

Your repository is:

</> Plain text

SE-Submissions_PES1UG24CS045

It currently has:

</> Plain text

```
Lab-1
lab_2
lab_03
```

We will add:

</> Plain text

```
lab_04
```

so it becomes:

</> Plain text

```
SE-Submissions_PES1UG24CS045/
|
├─ Lab-1/
├─ lab_2/
├─ lab_03/
|
└─ lab_04/
    ├─ before_video.mov
    ├─ after_video.mov
    ├─ chat_history.pdf
    └─ 45_snake/
        ├─ main.py
        ├─ requirements.txt
        ├─ README.md
        ├─ game/
        └─ assets/
```

The lab specifically says to push the completed work to your **personal repository** and **not raise a PR to the SETAPESU26 main repository**.



Lab_4_VibeCoding_Student_handou...



One thing before GitHub

Do the AFTER video now.

Don't spend time organizing GitHub yet.

Your immediate checklist is:

- ☒ Original game tested
- ☒ BEFORE video recorded
- ☒ Final game created
- ☒ Final game tested
- ☐ **AFTER video**
- ☐ Chat history PDF
- ☐ Create lab_04
- ☐ Put final code inside
- ☐ Push to GitHub
- ☐ Verify GitHub submission

 **So your next action is simply:**

Run the final game → record the AFTER video → save it as after_video.mov .

Once you've recorded it, tell me **"after video done"** and I'll give you the exact Terminal commands to create lab_04 and put everything into your GitHub repository safely.



Lab_4_VibeCodin...
PDF

I checked the **actual Lab 4 handout you uploaded**, so we can follow the teacher's requirements exactly.  Lab_4_VibeCoding_Student_handou...

Since your **final game is working**, you are almost finished.

 Your current status

| Requirement | Status |
|-----------------------|--|
| Clone assigned repo |  Done |
| Run original game |  Done |
| Record BEFORE video |  Done |
| Fix broken code |  Done |
| Add required features |  Done |
| Test final game |  Done |
| Record AFTER video |  Do this now |

| Requirement | Status |
|----------------------------|--------------------------|
| Export Claude chat history | ☐ |
| Prepare Lab-4 folder | ☐ |
| Add updated code | ☐ |
| Push to personal GitHub | ☐ |

The handout specifically requires another **10-second video after all changes**.

 Lab_4_VibeCoding_Student_handou...

So your immediate next step

Run your **new final game** and record:

1. Snake moving
2. Eating food / score increasing
3. Game Over screen
4. Replay/difficulty option
5. Starting a new game

Keep it around **10 seconds**.

Save it as:

 Plain text

after_video.mov

One correction about the folder name

Your teacher's handout says:

Upload the following to your Lab repo under **Lab-4 folder**.  Lab_4_VibeCoding_Student_handou...

So I recommend using:

 Plain text

Lab-4

rather than lab_04 , because **Lab-4** exactly matches the teacher's instruction.

Your final structure should be:

</> Plain text

```
SE-Submissions_PES1UG24CS045/
|
├─ Lab-1/
├─ lab_2/
├─ lab_03/
|
└─ Lab-4/
    |
    ├─ before_video.mov
    ├─ after_video.mov
    ├─ chat_history.pdf
    |
    └─ 45_snake/
        ├─ main.py
        ├─ requirements.txt
        ├─ README.md
        ├─ game/
        │   └─ game_engine.py
        │   └─ snake.py
        └─ food.py
    .
```

The handout's three required deliverables are exactly **before/after videos, updated code, and exported chat history**.  Lab_4_VibeCoding_Student_handou...

And remember: push it to **your personal repository** and **do not create a PR to SETAPESU26** .

 Lab_4_VibeCoding_Student_handou...

👉 **Right now:**

Record after_video.mov .

Once that is done, tell me **"after video done"** and I'll give you the exact Mac Terminal commands to create the final Lab-4 folder and push everything to your SE-Submissions_PES1UG24CS045 repository.

record after is done , what to do next

Excellent.  The **AFTER video is done**, so now only the submission packaging remains.

Your handout says the Lab-4 folder must contain **(a) before and after videos, (b) updated code, and (c) exported chat history as a doc/PDF**.  Lab_4_VibeCoding_Student_handou...

Step 1 — Get the chat history PDF

Since you used Claude for the coding work, export/download the **complete Claude conversation** that contains your prompts and Claude's responses.

Save it as:

</> Plain text

chat_history.pdf

Don't edit out the prompts/responses. The handout specifically asks for the chat history exported as a document/PDF.  Lab_4_VibeCoding_Student_handou...

Step 2 — Create your final Lab-4 folder

On your Desktop, create:

</> Plain text

Lab-4

Inside it, we want:

</> Plain text

```
Lab-4/  
├─ before_video.mov  
├─ after_video.mov  
├─ chat_history.pdf  
└─ 45_snake/
```

Put your videos inside

Copy your original BEFORE video into Lab-4 and rename it:

</> Plain text

before_video.mov

Copy your new AFTER video and rename it:

</> Plain text

after_video.mov

Step 3 — Put the FINAL code inside

Take the **final 45_snake_FINAL** project that you just tested.

Copy it into Lab-4 .

Rename the copied project folder:

</> Plain text

```
45_snake
```

So:

</> Plain text

```
Lab-4/  
└─ 45_snake/
```

Inside it should be the actual source:

</> Plain text

```
45_snake/  
├─ main.py  
├─ requirements.txt  
├─ README.md  
├─ game/  
│   ├── game_engine.py  
│   ├── snake.py  
│   └─ food.py  
└─ assets/  
    └─ ...
```

⚠ Don't copy these

Do **not** put these into your submission:

</> Plain text

```
.venv/  
venv/  
__pycache__/  
.git/  
.DS_Store
```

You only need the actual project/source files.

Step 4 — Final check

Before uploading to GitHub, your Desktop should look like:

</> Plain text

```
Lab-4/
|
|— before_video.mov      ✓
|— after_video.mov       ✓
|— chat_history.pdf      ✓
|
|— 45_snake/             ✓
    |— main.py
    |— requirements.txt
    |— README.md
    |— game/
    |— assets/
```

This directly covers the three deliverables in your handout.  Lab_4_VibeCoding_Student_handou...

Step 5 — Then put it in your GitHub repository

Your personal repository is:

</> Plain text

```
SE-Submissions_PES1UG24CS045
```

We want:

</> Plain text

```
SE-Submissions_PES1UG24CS045/
|
|— Lab-1/
|— lab_2/
|— lab_03/
|
|— Lab-4/
    |— before_video.mov
    |— after_video.mov
```

```
├─ chat_history.pdf
└─ 45_snake/
    ├── main.py
    ├── requirements.txt
    ├── README.md
    ├── game/
    └── assets/
```

The handout says to push the code to **your personal repo**, and not to raise a PR to SETAPESU26 .

 Lab_4_VibeCoding_Student_handou...

Don't push yet

First, let's make sure the folder is correct.

Your next action:

Create the **Lab-4** folder and put these **three things** inside:

 Plain text

1. before_video.mov
2. after_video.mov
3. chat_history.pdf

Then put your **final tested 45_snake code folder** inside it.

When that's done, send me a **screenshot of the Lab-4 folder**.

I'll then give you the exact Git commands to put it into SE-Submissions_PES1UG24CS045 and push it safely.

[how to get chat history from claude](#)

Yes. Since your teacher requires the **chat history exported as a document/PDF**, the easiest official way is to export your Claude data.  Lab_4_VibeCoding_Student_handou...

Fastest method on Claude Web

1. Open **Claude** on your browser.
2. Click your **initials/profile** in the **bottom-left corner**.
3. Click **Settings**.
4. Open **Privacy**.
5. Click **Export data**.

6. Claude will process the export and send a **download link to your account email**.

🌐 Claude Help Cent...

7. Download the export when the email arrives.

The official Claude documentation says the export includes your **conversation data and user data**.

🌐 Claude Help Cent...

⚠️ But for your assignment

You probably **don't want to submit your entire Claude account data export** if it contains all your other conversations.

You need the conversation where you did this project:

</> Plain text

45_snake.zip

↓

Claude prompts

↓

Collision fix

↓

Game Over

↓

Replay/Difficulty

↓

Sound

↓

Final 45_snake_FINAL.zip

Better option for your Lab-4 PDF

Open the specific Claude conversation and click **Share** in the upper-right. Claude can create a shareable snapshot containing the conversation messages up to that point. 🌐 Claude Help Cent...

Then, if you need an actual PDF:

Mac → open the chat → ⌘ + P → PDF → Save as PDF

Name it:

</> Plain text

chat_history.pdf

This gives you a clean PDF of the **actual project conversation**, which is much more suitable for your Lab-4 submission.

Your final Lab-4 folder will then be:

</> Plain text

```
Lab-4/
├── before_video.mov
├── after_video.mov
├── chat_history.pdf
└── 45_snake/
    ├── main.py
    ├── requirements.txt
    ├── README.md
    ├── game/
    └── assets/
```

I recommend the **specific-chat → Print → Save as PDF method** rather than submitting Claude's entire account export.

If you tell me whether you're using **Claude in Chrome on your Mac**, I can give you the exact clicks to save that conversation as `chat_history.pdf`.

[how to get in chat got](#)

If you mean **how to get the Claude chat history for your Lab-4 submission**, do this:

Easiest method — save the specific Claude chat as PDF

1. Open **Claude** on your Mac.
2. Open the **chat where you worked on the Snake project**.
3. Make sure the entire conversation is there — your prompts + Claude's responses.
4. Click **Share** in the **top-right corner**.
5. Click **Share** again to create the chat snapshot. Claude says the snapshot includes the messages sent before sharing. [Claude Help Cent...](#)
6. Open the shared chat/snapshot in your browser.
7. Press **⌘ + P** on your Mac.
8. In the print window, select **Save as PDF**.
9. Save it as:

</> Plain text

```
chat_history.pdf
```

10. Put that PDF inside:

</> Plain text

```
Lab-4/
├─ before_video.mov
├─ after_video.mov
├─ chat_history.pdf
└─ 45_snake/
```

This is suitable for your assignment because the handout specifically asks for the **chat history exported as a DOC/PDF**.  Lab_4_VibeCoding_Student_handou...

⚠️ Don't use "Export Data" for this

Claude also has **Settings → Privacy → Export data**, but that exports your account's conversation/user data rather than just this one project chat.  Claude Help Cent...

For your **Lab-4 submission**, the **specific-chat → PDF** method is much cleaner.

If you want, I can also show you **exactly how your final Lab-4 folder should look before you push it to GitHub**.

chat gpt history

Ah, you mean **ChatGPT chat history**, not Claude. 👍

For your **Lab-4 assignment**, if you want to save **this ChatGPT conversation as a PDF**, the easiest way is:

Option 1 — Save this particular chat as PDF

1. Open the ChatGPT conversation containing your Lab-4 work.
2. Scroll through the conversation and make sure the required prompts/responses are loaded.
3. Press **⌘ + P** on your Mac.
4. In the print window, choose **Save as PDF**.
5. Name it:

</> Plain text

```
chat_history.pdf
```

6. Save it inside your **Lab-4** folder.

Your folder will then be:

</> Plain text

```
Lab-4/
├─ before_video.mov
```

- |— after_video.mov
- |— chat_history.pdf
- |— 45_snake/

This is enough if your teacher simply wants the **ChatGPT conversation as a PDF**.

Option 2 — Export your complete ChatGPT history

If you want the official ChatGPT data export:

Profile → Settings → Data Controls → Export → Confirm export

OpenAI will send you a download link. The ZIP contains your chat history along with other account data.  OpenAI Help Cen...

For your Lab-4 submission, I recommend Option 1 because you only need the relevant project conversation, not your entire ChatGPT account history. The assignment specifically asks for the chat history exported as a DOC/PDF.  Lab_4_VibeCoding_Student_handou...

If you want, I can also tell you **exactly which ChatGPT conversation/messages you should include in chat_history.pdf** for the Snake assignment.